



PhishGuard-AI

From admin dashboard to Security Operations Centre

Technical report — phase 3 (administration, cyberdefence, real-time operations)

Item	Detail
Scope	Admin security centre, user approval & sign-up intelligence, live system analysis, allowlist with logos and health, automatic report triage (80 %), real incidents, real threat intelligence, audit trail, bilingual UI
Stack	Flask 3 + PostgreSQL · Vue 3 + TypeScript + Tailwind v4 · psutil · OpenPhish feed · ip-api.com
Verification	vue-tsc + Vite build OK · 31 backend tests OK · end-to-end smoke tests of every new flow
Date	25 September 2026

1. Executive summary

The admin area was a set of attractive but partly simulated screens. It is now a working **Security Operations Centre**: every number comes from the database or from live measurements, every sensitive action is protected and traced, and the platform reacts automatically to threats (triage, correlation, blocking) while leaving the final word to humans.

Requirement	What was built
Better security & settings for the admin	Three-tab settings: identity card (photo, name, job title, organisation, contact), personal security centre (posture score A–D with fix-it checklist, MFA, password policy meter, login history with IP/place/device, sign out other sessions) and platform policies
Advanced admin security (creative)	Sudo mode (step-up re-authentication) for sensitive actions, real session revocation (session versioning), idle timeout, MFA removal protected by MFA, new-location login detection, audit of every admin action
User management with approval	Approval workflow (approve / verify further / reject, with a note shown to the user), registration intelligence (IP, city, ISP, VPN/datacentre flags, device, user-agent, terms timestamp), sign-up risk score , accounts sharing an IP, activity timeline, account controls
More information at registration	Full name (required), region, city; hidden bot trap; fill-time measurement; disposable-email refusal
Real system analysis	System page + command-centre band: API rpm / error rate / p50-p95, CPU-RAM-disk, database latency/size/connections/row counts, engine components, pipeline durations, AI coverage — refreshed live
Allowlist with logos, live, CRUD	Institution cards with logo (site favicon or custom), add / edit / enable / delete, live DNS + TLS + certificate expiry + latency checks, auto-refresh
Reports handled at 80 %	Automatic triage: the engine proposes label + confidence; assisted mode closes items with confidence $\geq 80\%$ (adjustable), manual and automatic modes, bulk acceptance, engine accuracy measured on human decisions
Real incidents / reports / threat intel	Correlation engine opening incidents from real detections, case board with playbook, timeline, notes, assignment, MTTR; IOC database from real messages, OpenPhish cross-check, targeted brands, AI threat categories, blocklist used by the detection pipeline, real audit log

2. Architecture

The design follows one rule: **one module, one responsibility**, and the API layer only orchestrates services. Everything new is additive; the detection pipeline contract did not change.

Data flow

scan → pipeline (ML + AI + URL intel + rules + **blocklist**) → analysis stored → triage_service (uncertain / reported) → correlation_service (incidents) → admin screens · every action → audit_service → security_events

Layer	Module	Responsibility
Database	08_soc_platform.sql	users (+approval, registration intel, risk, session_version), security_events, incidents, blocked_domains, allowlist logo/health, triage columns
Service	client_info.py	Real client IP (proxy-aware, anti-spoofing), device label, IP geolocation
Service	audit_service.py	Single entry point for security & audit events; never breaks the audited action
Service	registration_risk.py	Explainable sign-up risk scoring and approval decision
Service	triage_service.py	Suggestion + confidence; manual / assisted / auto policies; accuracy stats
Service	indicators.py / correlation_service.py	IOC extraction; incident correlation, severity, timeline, blocking
Service	system_metrics.py	Request ring buffer, host (psutil), database and engine telemetry
Service	domain_health.py / threat_feed.py	Allowlist DNS/TLS checks; OpenPhish feed with cache and back-off
API	security.py	@sudo_required step-up decorator
API	admin_security.py	Posture, sudo, sessions, audit events, alerts
API	admin_soc.py	System, incidents, threat intel, blocklist
API	admin.py / admin_ops.py / auth.py	Users & approval; triage & allowlist; hardened registration and login
Frontend	soc.service.ts, http.ts	Typed SOC client; transparent sudo prompt + replay
Frontend	components/admin/ui/*	Design kit: PageHeader, Panel, StatTile, Pill, Sparkline, RingGauge
Frontend	views/admin/*	Settings, UserManagement, Whitelist, ReviewQueue, Incidents, ThreatIntelligence, AuditLog, Health (System)

3. Security engineering (cyberdefence view)

Controls added, mapped to the threats they address:

Threat	Control implemented
Stolen session cookie	Session versioning: force logout / password change / suspension invalidate every cookie at once; idle timeout; sudo mode for destructive actions
Stolen admin password	MFA in the security centre, sudo requires password <i>and</i> TOTP, MFA cannot be removed without a valid code, new-location logins flagged
Credential stuffing / brute force	Lockout policy, every failure logged with IP and place, locked accounts counted and unlockable by an admin
Account enumeration	Password checked before any account state is revealed; identical answer and timing for unknown accounts
Fake / bot sign-ups	Honeypot field (bots get a fake success, nothing is created), fill-time check, disposable-email refusal, IP reuse & sign-up wave detection, VPN/datacentre flags, risk-based approval
IP spoofing in logs	X-Forwarded-For trusted only from the local proxy
Information leakage	Generic error messages instead of raw exceptions in auth endpoints
Insider / admin abuse	Immutable audit trail of every admin action (who, what, when, IP, place, device), CSV export for investigations
Phishing campaigns	Automatic correlation into incidents, one-click platform-wide domain blocking enforced by the detection pipeline, response playbook, OpenPhish cross-check
Automation errors	Human decisions always override the engine; automatic decisions are labelled and excluded from the retraining export (no self-poisoning)

Automatic triage — how the 80 % works

confidence = distance of the score from the 50 decision line × agreement of the independent signals (ML, AI, URL, rules) ± the AI's own confidence (bonus if it agrees, penalty if not); ≥ 92 when hard evidence (blocklist / brand spoofing) exists; reduced when a user reports a message the engine found safe. In **assisted** mode only items with confidence ≥ threshold (default 80 %) are closed automatically; the rest stays for an analyst. The engine's real accuracy is measured continuously against human decisions and shown on the queue.

Incident correlation

Dangerous messages of the last 72 h are grouped by registered domain and impersonated brand; from 3 messages (configurable) an incident opens with a severity derived from max score, volume and number of victims, and every later message extends its timeline.

4. Verification

- Backend: 31 existing tests pass; every new endpoint called as an admin (smoke test).
- Registration: disposable email refused; bot trap returns a fake success and creates nothing; real Cameroonian IP resolved (Douala, Orange Cameroun, mobile); approval gate blocks login until approval; session revocation returns 401.
- Sudo: deletion refused without sudo, wrong password refused, allowed after confirmation.
- Incident flow: manual incident → domain blocked → a new scan containing that domain is forced to phishing (score 85) → note → resolved; full admin audit trail recorded.
- Correlation on existing data opened real incidents; OpenPhish feed downloaded (≈300 live phishing URLs).
- Allowlist health: afrilandfirstbank.com, mtn.cm, orange.cm online with certificate lifetimes (fallback to www. for sites without HTTPS on the bare domain).
- Frontend: vue-tsc type-check and production build pass; every translation key present in French and English.
- Not yet done: a visual walkthrough in a browser of every new screen in both themes and languages.

5. Recommendations / next steps

- Turn on MFA for the admin account now (posture currently D) and enable “MFA required for everyone”.
- Deploy behind HTTPS and set SESSION_COOKIE_SECURE=True; replace the placeholder SECRET_KEY; rotate exposed keys.
- Add the scanner of real mailboxes (Gmail/Outlook OAuth, see phase-2 report) — the SOC is ready to receive its alerts.
- Schedule correlation, allowlist health checks and the OpenPhish refresh as background jobs (APScheduler/Celery).
- Retrain the text and URL models on the human-labelled export (automatic decisions are already excluded).
- Add rate limiting (Flask-Limiter) on login, registration and scan endpoints; add automated tests for the new services.